

NeuroPrefetcher

Storage-Aware Sparse LLM Inference via Delta Prefetching

Nobel Dhar, Md Romyull Islam, Xuechen Zhang, Gongjin Sun[†], Sahidul Islam, Bobin Deng, Kun Suo

Kennesaw State University [†]Memory Solutions Lab, Samsung Semiconductor, Inc.

ICPP '26 · Singapore · Session 4A: LLM Inference and Serving III

Edge LLMs are outgrowing edge memory

Mistral-7B (FP16) on Jetson AGX Orin, mem = 14G

Model weights **13.49 GiB**



Available to CUDA **11.9 GiB**



1.6 GiB short

mem = 14G caps the 32 GiB board at 14 GiB, as when the LLM shares the device; after the OS, display and CUDA runtime, 11.9 GiB is left. The model never fits: storage must supply weights during inference.

~10× vs ~2×

model size per generation vs. edge DRAM capacity

- Inference memory is a deployment parameter, not a hardware constant
- Perception pipelines, apps and system services share the same device
- Storage becomes an **active weight tier**

Existing answers assume the model will fit

1 Quantization

GPTQ, AWQ: a 7B model shrinks from 13.5 GiB to ~4 GiB at 4-bit

Moves the fit point; a 4-bit model can still exceed memory

2 Smaller models

Phi-3, MobileLLM: designed for edge budgets

Trades model capability for a fixed memory budget

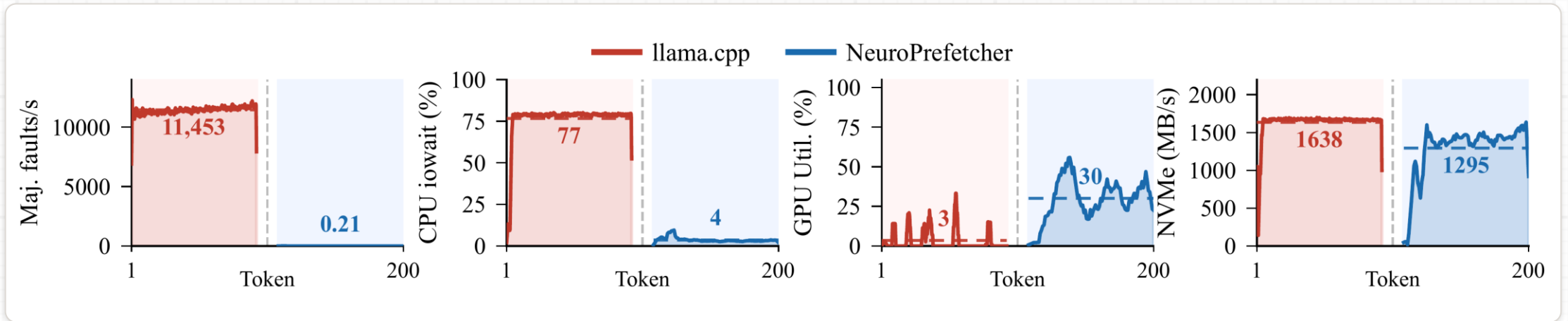
3 Offloading

llama.cpp, FlexGen: spread weights over GPU, CPU memory and storage

Weights move reactively, when execution reaches them

Our target: the model stays larger than resident memory for the entire run. Performance then depends on how efficiently weights move from storage to compute.

Reactive paging leaves the GPU idle



llama.cpp (mmap + OS demand paging) vs. **NeuroPrefetcher** on the same board, Mistral-7B, mem = 14G

Major page faults / s

11,453 → **0.21**

llama.cpp → NeuroPrefetcher

CPU iowait

77% → **4%**

llama.cpp → NeuroPrefetcher

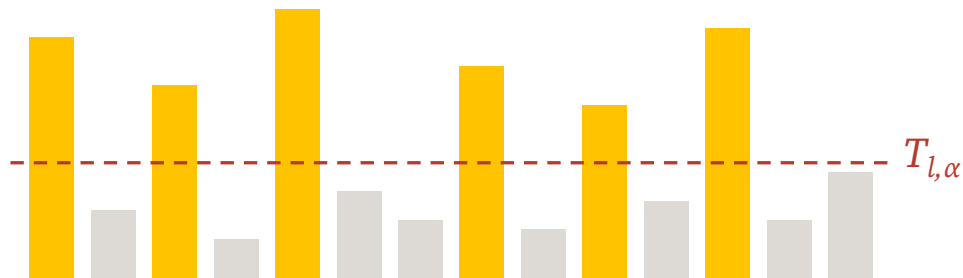
GPU utilization

3% → **30%**

llama.cpp → NeuroPrefetcher

Two properties we can exploit

1 Activation sparsity

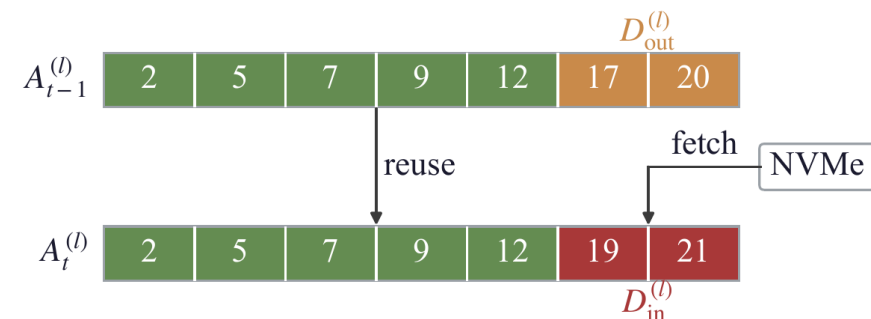


- Per token, many MLP neurons contribute little to the output
- SwiGLU models (Mistral, Llama) can drop small activations with limited quality loss
- In a storage-backed runtime, sparsity cuts **bytes read**, not just FLOPs

2 Temporal locality

82-85%

of active MLP neurons
persist from one token
to the next



Most of the next token's sparse weights are already resident: only the **delta** must come from storage

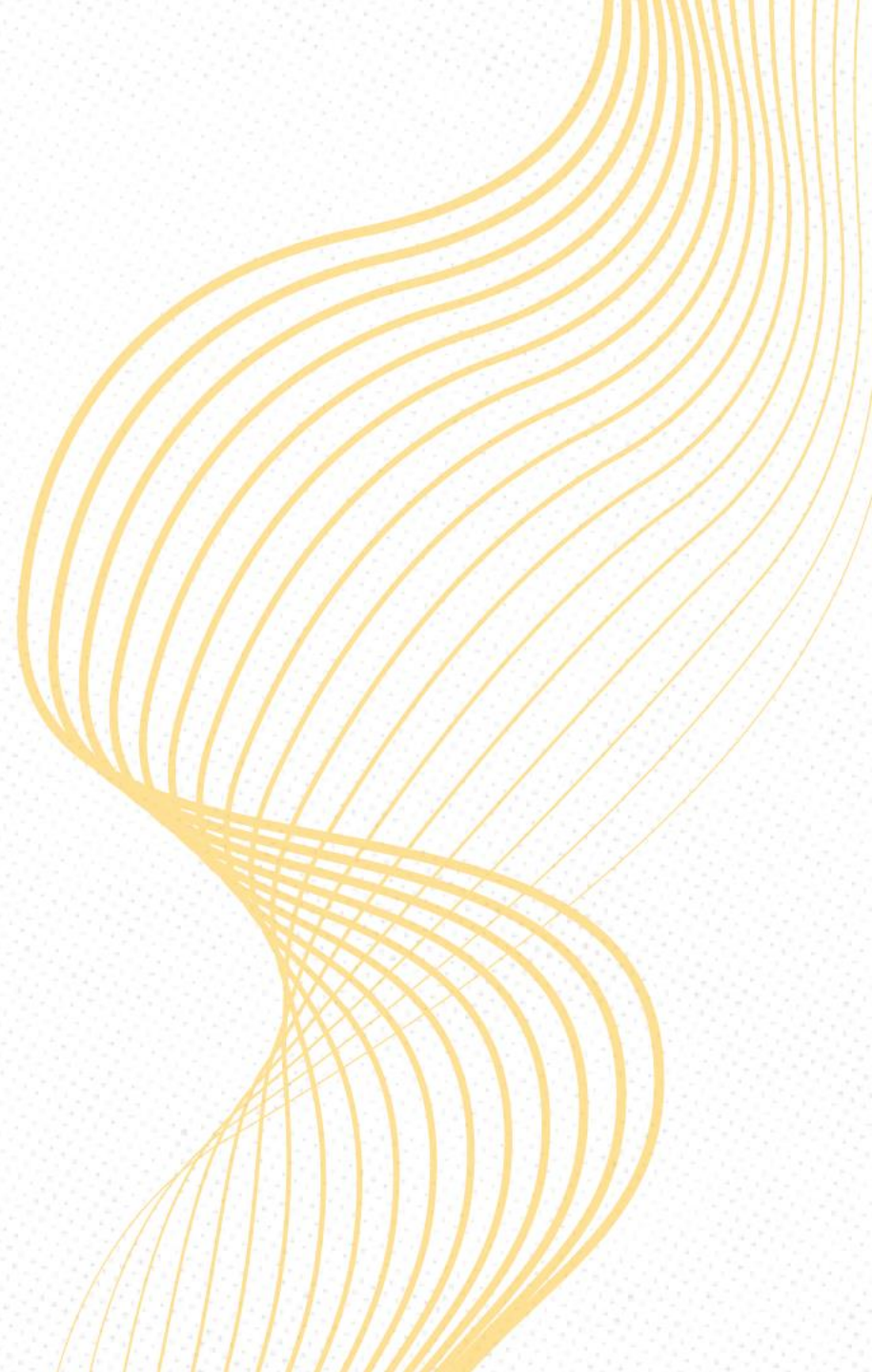
What prior sparse runtimes are missing

Exploiting locality needs both: (1) track which rows are already resident per layer, and (2) know the next active set **before** execution reaches the layer.

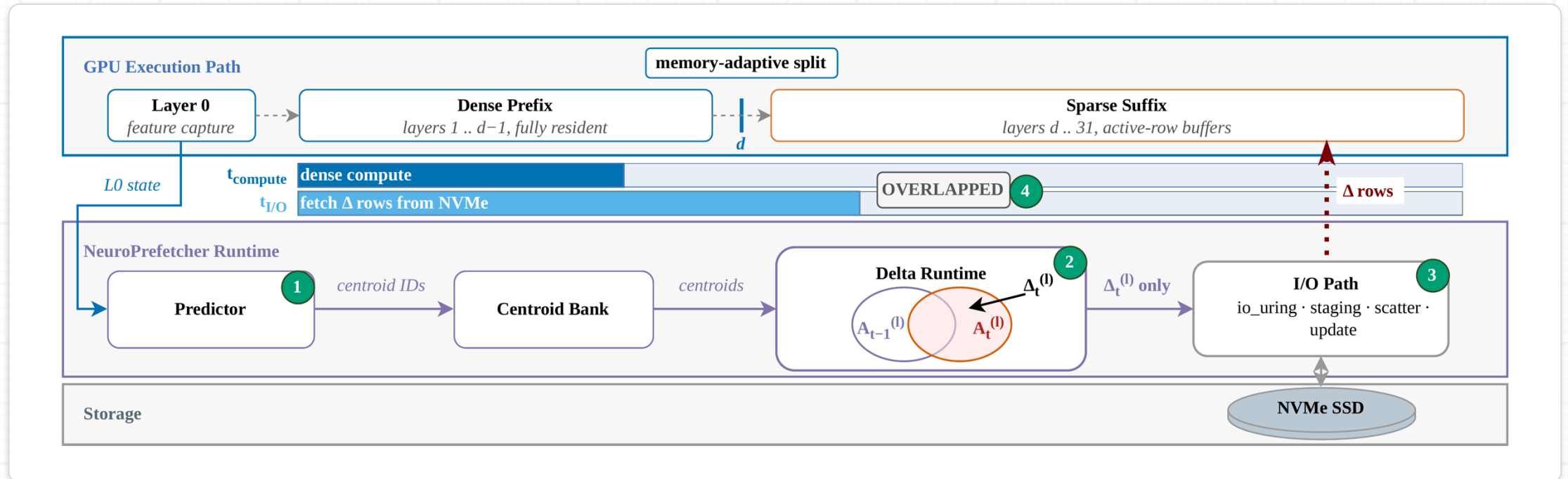
System	Next active set known...	What it optimizes
DejaVu	Per layer, only when execution reaches it	Computation
PowerInfer	Offline hot/cold profile + per-layer prediction	Hot neurons on GPU, cold on CPU (GPU ↔ CPU memory)
LLM in a Flash	No lookahead: backward-looking window	Retains recently activated neurons
Neuralink	No lookahead	Co-locates co-activated neurons in flash
NeuroPrefetcher	After layer 0, for all layers (lookahead)	Only incoming delta rows, NVMe → GPU

Design

Predict once
Fetch only the delta
Adapt to memory



NeuroPrefetcher at a glance



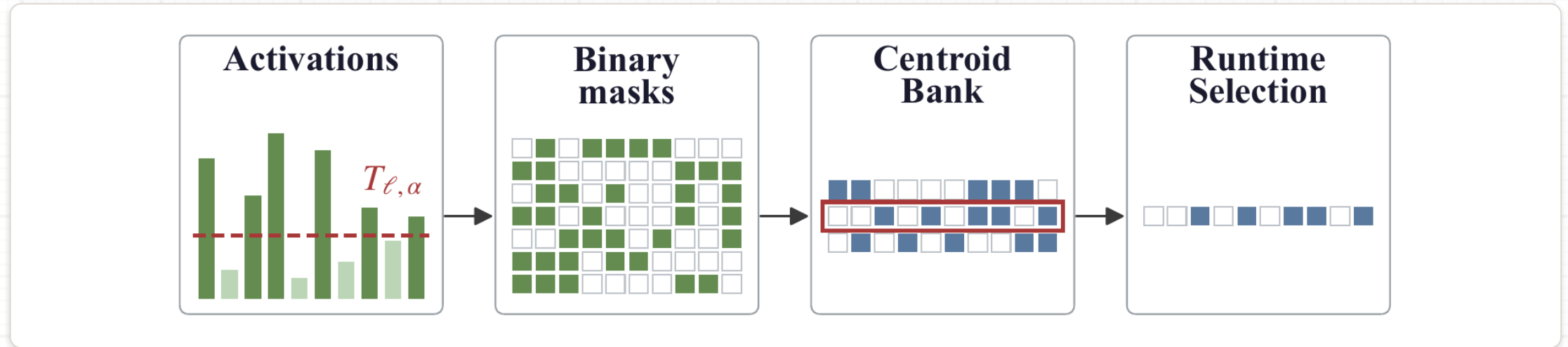
1 Predictor: after layer 0, pick centroids for all later layers

2 Delta runtime: diff the predicted set against resident rows

3 I/O path: read only the delta rows from NVMe, stage, scatter

4 Overlapped: I/O runs while the GPU computes the dense prefix

Offline: sparse centroids as a pattern vocabulary



1 Threshold

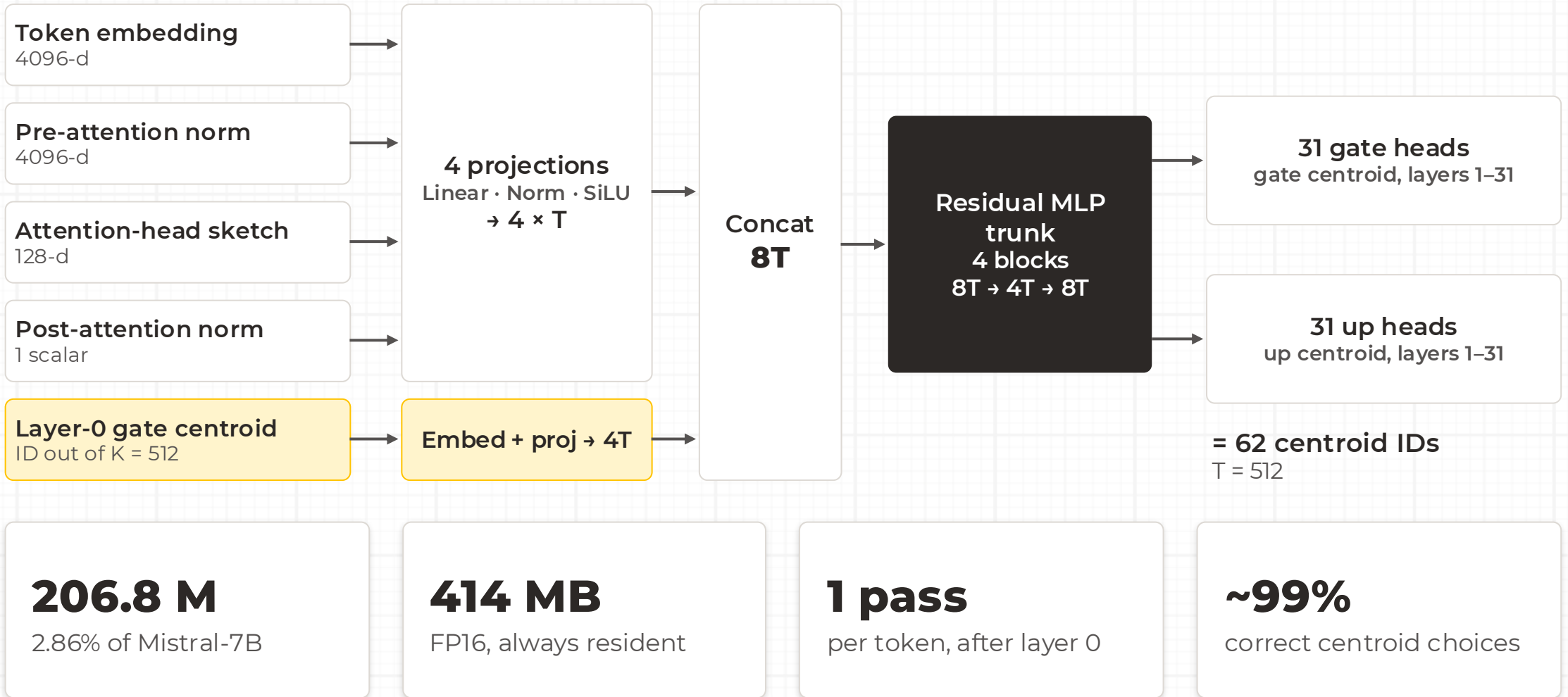
- Per layer l , zero activations below the α -th percentile $T_{l, \alpha}$ of $|A|$
- No retraining; α sets the active-neuron budget

2 Cluster

- Group similar binary masks; aggregate each group and prune back to the target sparsity
- **K = 512** centroids: one choice per layer instead of 14,336 neuron decisions

One predictor, all layers, once per token

LAYER-0 FEATURES



Predictor inputs and cost

Inputs, all taken from layer 0

Feature	Input form
Token embedding	4096-d vector
Pre-attention norm	4096-d vector
Attention-head sketch	128-d vector
Post-attention RMS norm	1 scalar
Gate projection structure	centroid ID

Layer-0 gate activations enter as one centroid ID instead of the full 14,336-d vector.

Parameters (FP16)

Component	Params	Size (MB)
Input projections	4.3M	8.6
Centroid embedding	5.2M	10.4
Residual trunk	67.1M	134.2
Output heads	130.2M	260.4
Total	206.8M	413.6

Heads grow with layer count; the trunk is fixed; nothing scales with MLP width. All speedups are net of this cost.

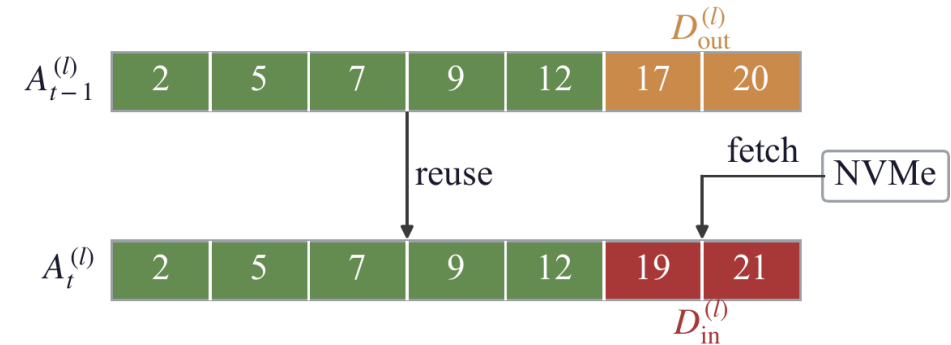
Predictive delta prefetching

Per sparse layer l at token t

$$D_{in} = A_t \setminus A_{t-1} \quad \text{fetch from NVMe}$$

$$D_{out} = A_{t-1} \setminus A_t \quad \text{free the slot}$$

- A_t = gate mask n up mask: a SwiGLU neuron contributes only if both are active
- Fixed-capacity buffer of K_{max} rows; freed slots are reused, no compaction
- First token loads the full predicted set; every later token uses the delta path

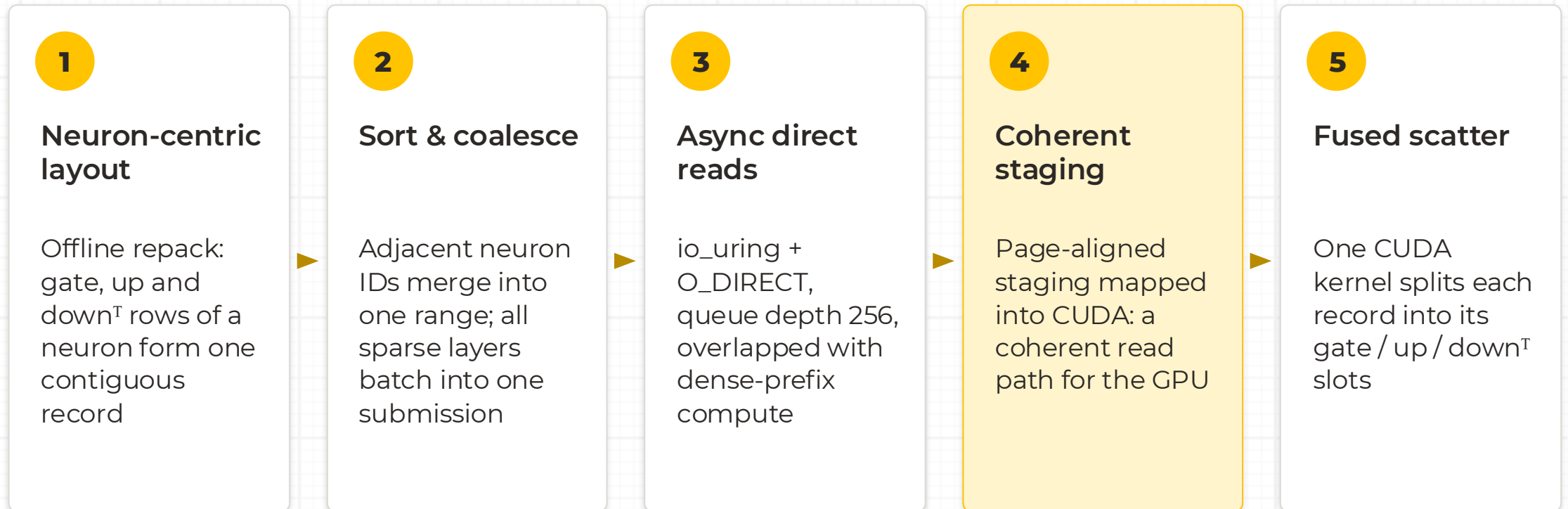


Slot map per layer

n	2	5	12	17	19
S[n]	0	1	4	-1	5

neuron $\rightarrow$ buffer slot (-1 = not resident); the sparse kernel only needs presence + slot

I/O path: model-aware sparse-row transfers



One read returns everything an active neuron needs. Only step 4 (mapped staging via `cudaHostRegisterMapped`) is Jetson-specific.

Memory-adaptive dense/sparse split

32 transformer layers of Mistral-7B



Dense prefix: d layers
MLP weights fully resident

Sparse suffix: $32 - d$ layers, weights on NVMe
 K_{max} -row buffer per layer: 129 MiB (62%) / 231 MiB (31% sparsity)

Rising

Each extra dense layer removes NVMe traffic and widens the compute window that hides I/O

Peak d^*

Largest dense prefix that still keeps every sparse layer's buffer resident

Falling

Past d^* , buffers are squeezed out and layers fall back to whole-layer NVMe reads

d is calibrated once per device and sparsity. At 62% sparsity: $d = 2$ (9.0 GiB) $\rightarrow 10$ (11.0) $\rightarrow 18$ (13.2) $\rightarrow 28$ (14.8 GiB)

Evaluation

Jetson AGX Orin

Mistral-7B · Llama-3-8B

Throughput · I/O · latency · quality



Experimental setup

1 Platform

- NVIDIA Jetson AGX Orin dev kit
- 32 GiB unified LPDDR5
- 12-core A78AE, Ampere GPU
- 990 PRO NVMe SSD, 7.4 GB/s
- mem= 11G-17G → 9.0-14.8 GiB

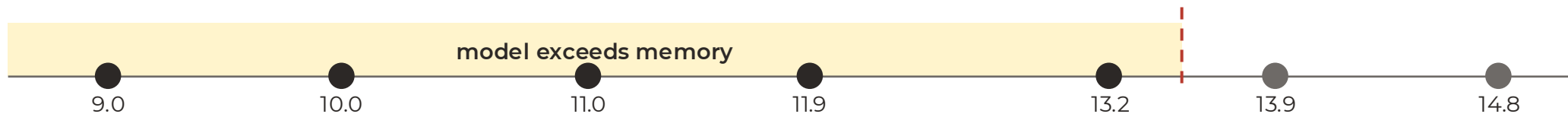
2 Models & sparsity

- Mistral-7B-v0.1, FP16, 13.49 GiB: all experiments
- Llama-3-8B: quality study
- Sparsity: 62% / 50% / 31%
- $K_{max} = 5,500 / 7,168 / 9,865$

3 Methodology

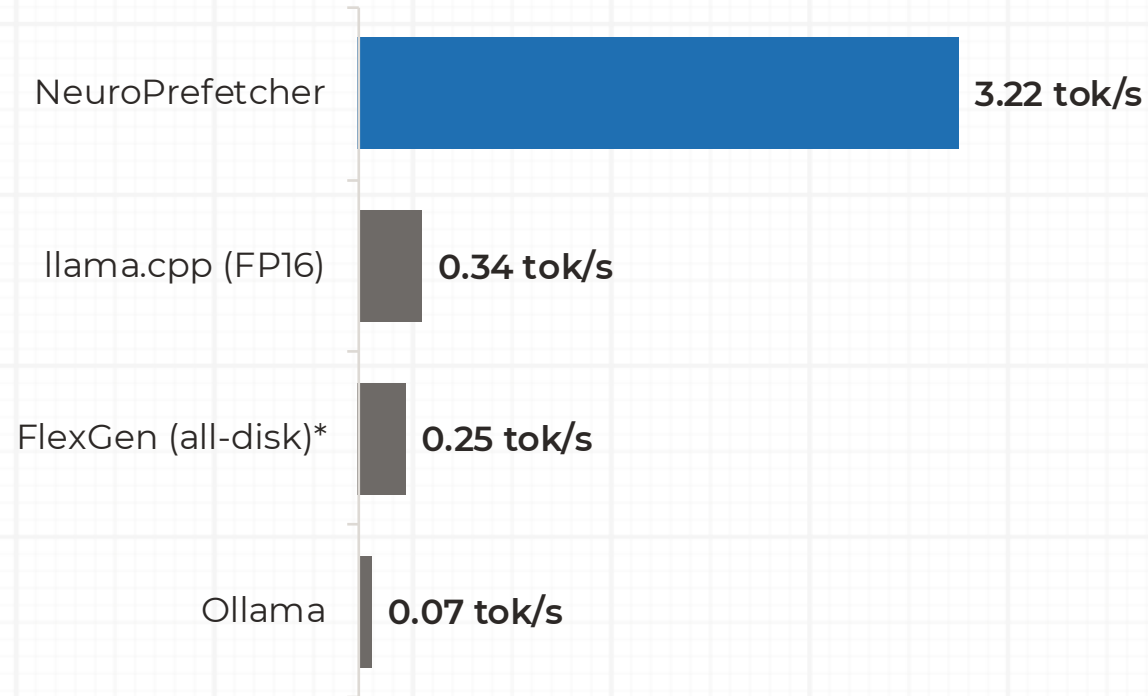
- Decode tok/s over 2,048 tokens (WikiText-2)
- WikiText-2 perplexity, 10-shot HellaSwag
- Baseline: llama.cpp FP16, mmap
- Offline prep on 8× A100

Memory available to CUDA (GiB)



Capability: only 4 of 20 systems produce output

Decode throughput at mem = 14G (11.9 GiB for CUDA)



NeuroPrefetcher: 9.5× llama.cpp. *FlexGen runs OPT-6.7B (12.4 GiB); no feasible mixed placement on unified memory

Allocation failure (6)

HF Transformers, HF Transformers (disk),
ZeRO-Inference, CATS, TEAL, SGLang

Weight transfers exceed unified-memory headroom

Unified-memory mismatch (6)

vLLM, TensorRT-LLM, MLC-LLM,
PowerInfer, Candle, DejaVu

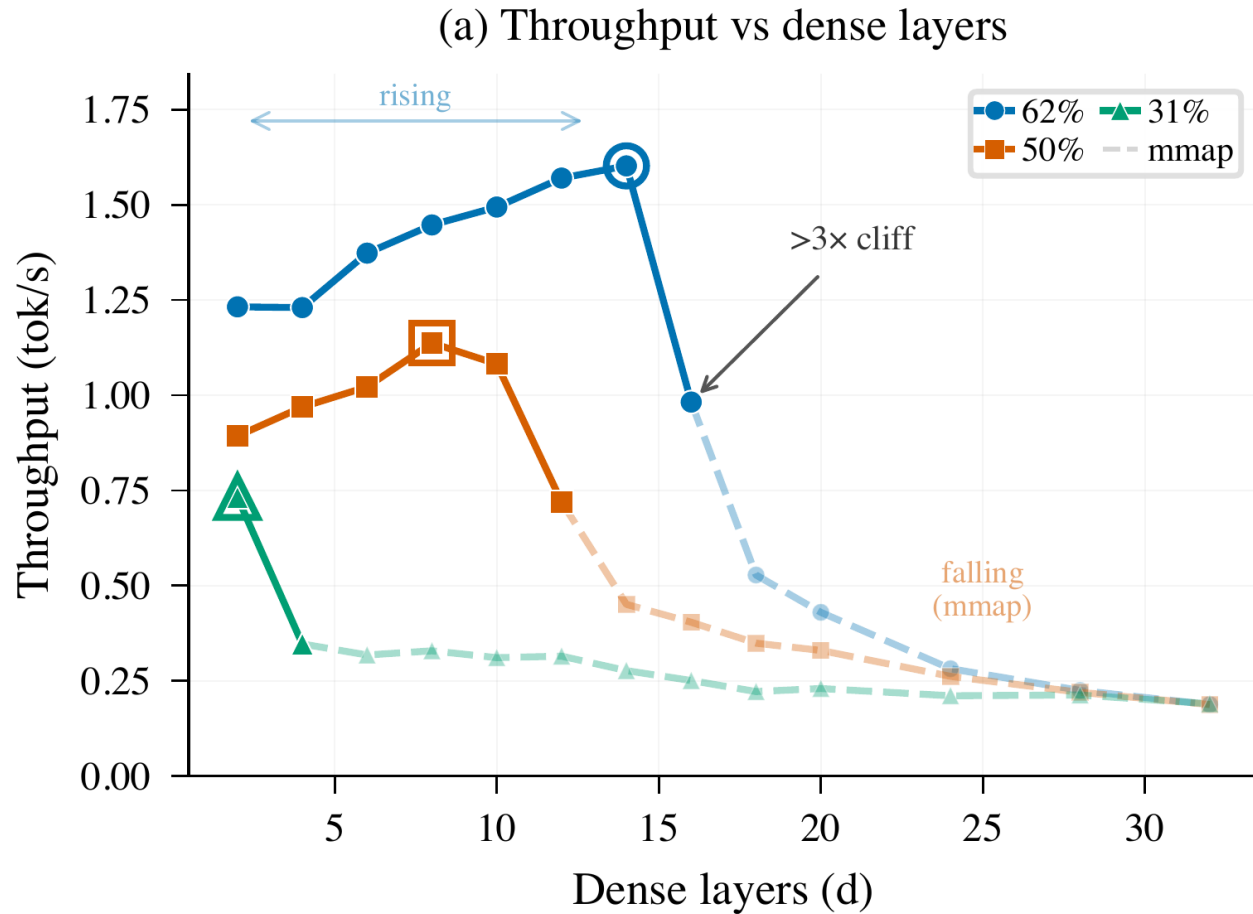
Assume separate GPU and host memory tiers

No ARM / JetPack release (4)

LLM in a Flash, LMDeploy, TGI,
HF Optimum + ONNX GPU

No deployable 64-bit ARM build

Choosing d^* : the dense/sparse sweep



Rising

62%: 1.23 $\rightarrow$ 1.60 tok/s from $d = 2$ to $d = 14$

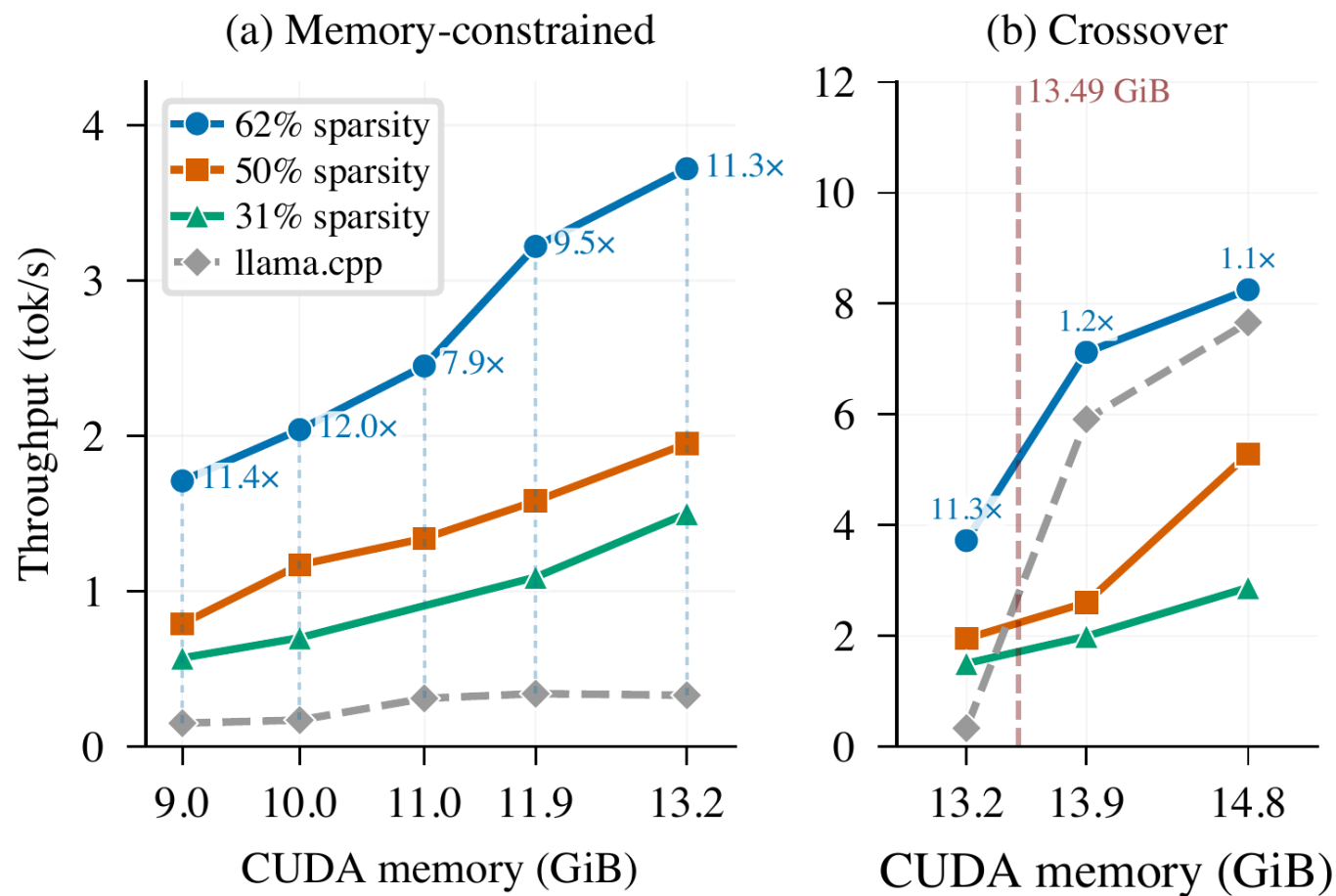
Peak d^*

$d^* = 14 / 8 / 2$ at 62% / 50% / 31% sparsity

Falling

$d = 16$ loses two buffers: 1.60 $\rightarrow$ 0.98 tok/s

7.9–12.0× faster when the model exceeds memory



7.9–12.0×

over llama.cpp, 9.0–13.2 GiB
(62% sparsity)

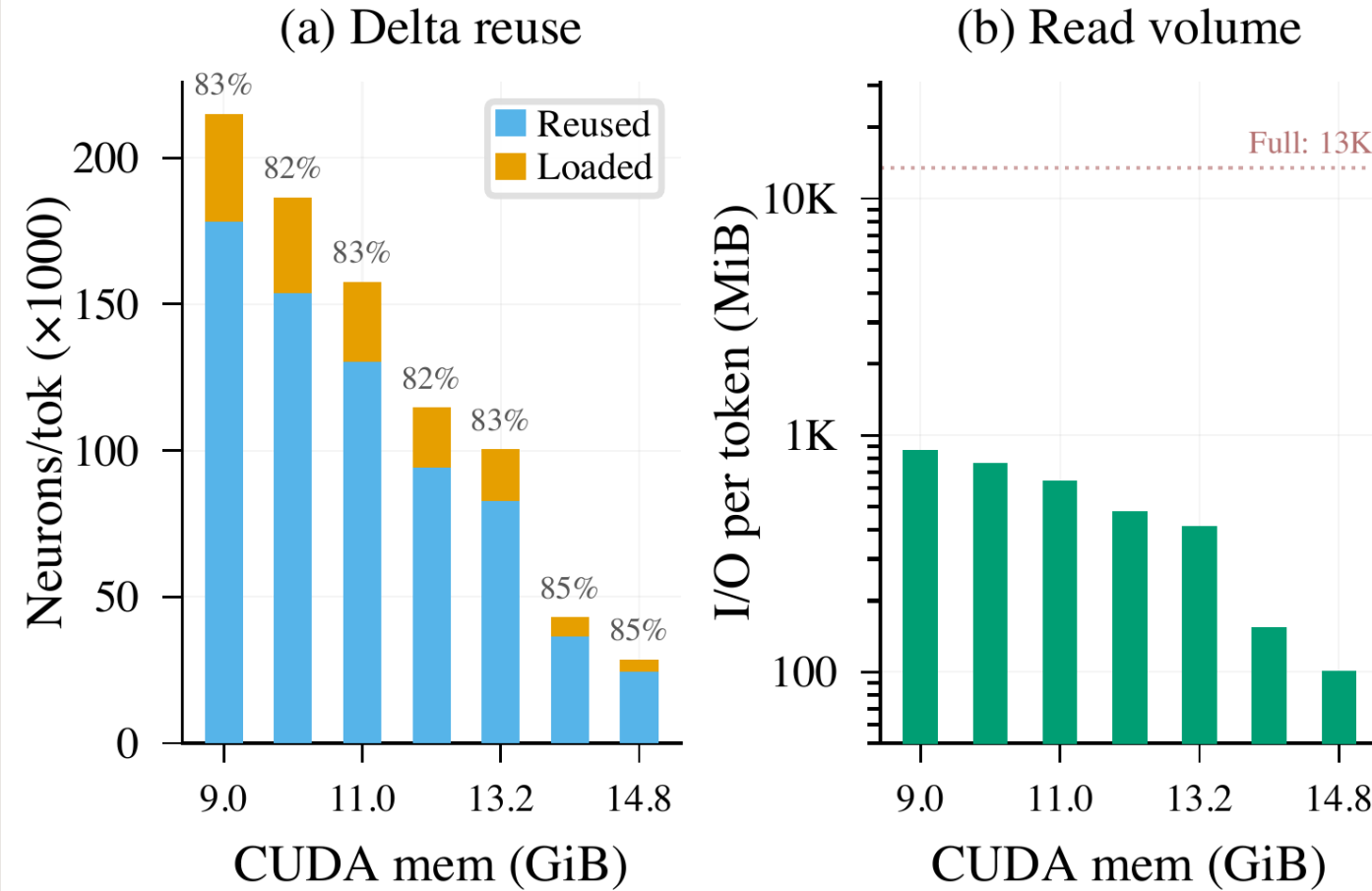
3.22 vs 0.34

tok/s at 11.9 GiB for CUDA
(mem=14G)

1.2× / 1.1×

once the model fits (13.9 / 14.8 GiB)

Delta reuse keeps NVMe traffic small



82–85%

of sparse rows reused from the previous token (62% sparsity)

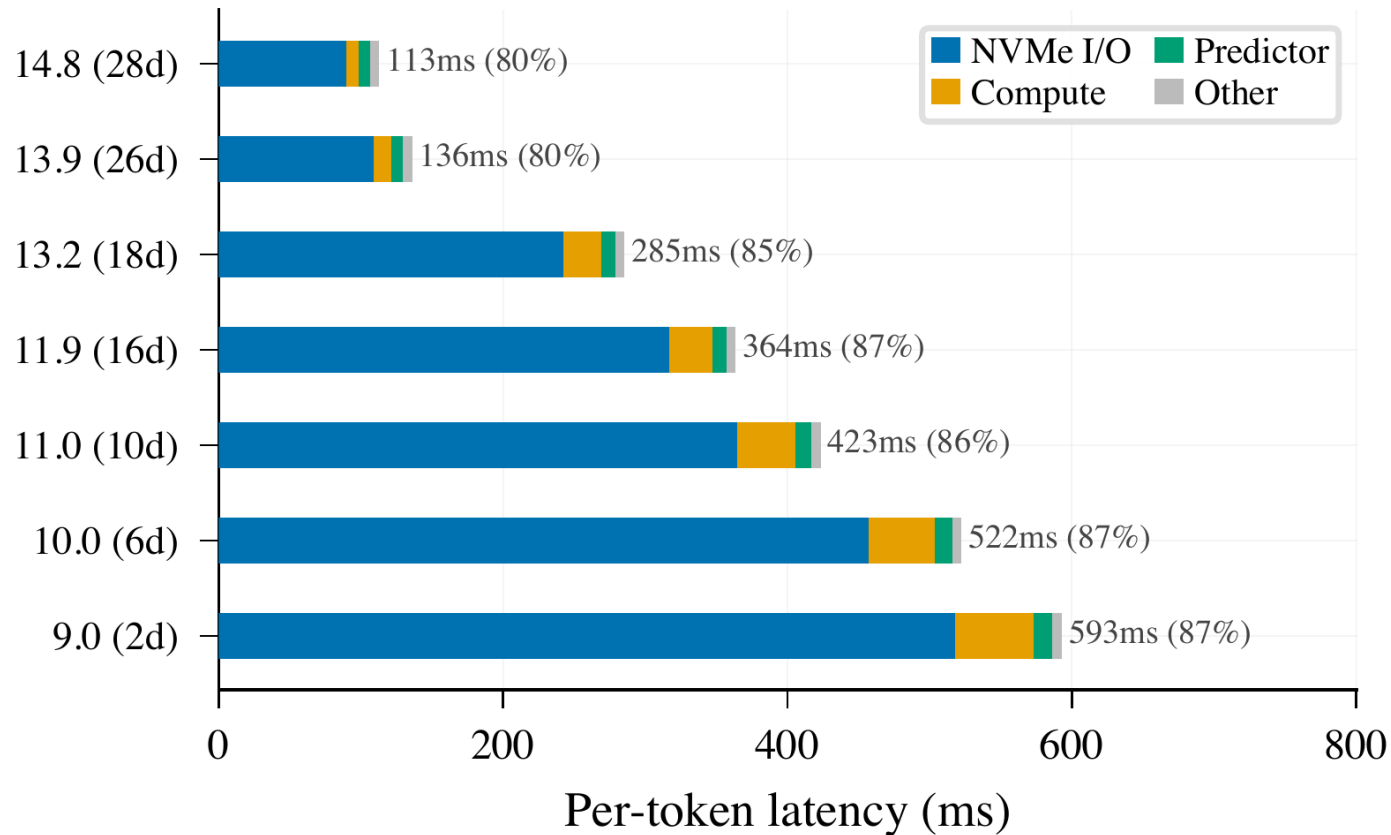
15–18%

of needed rows actually cross the NVMe boundary

<1 GiB

per token at 9.0 GiB (103 MiB at 14.8 GiB) vs. 13.49 GiB model

Where the time goes



80–87%

of per-token latency is NVMe I/O at every memory level

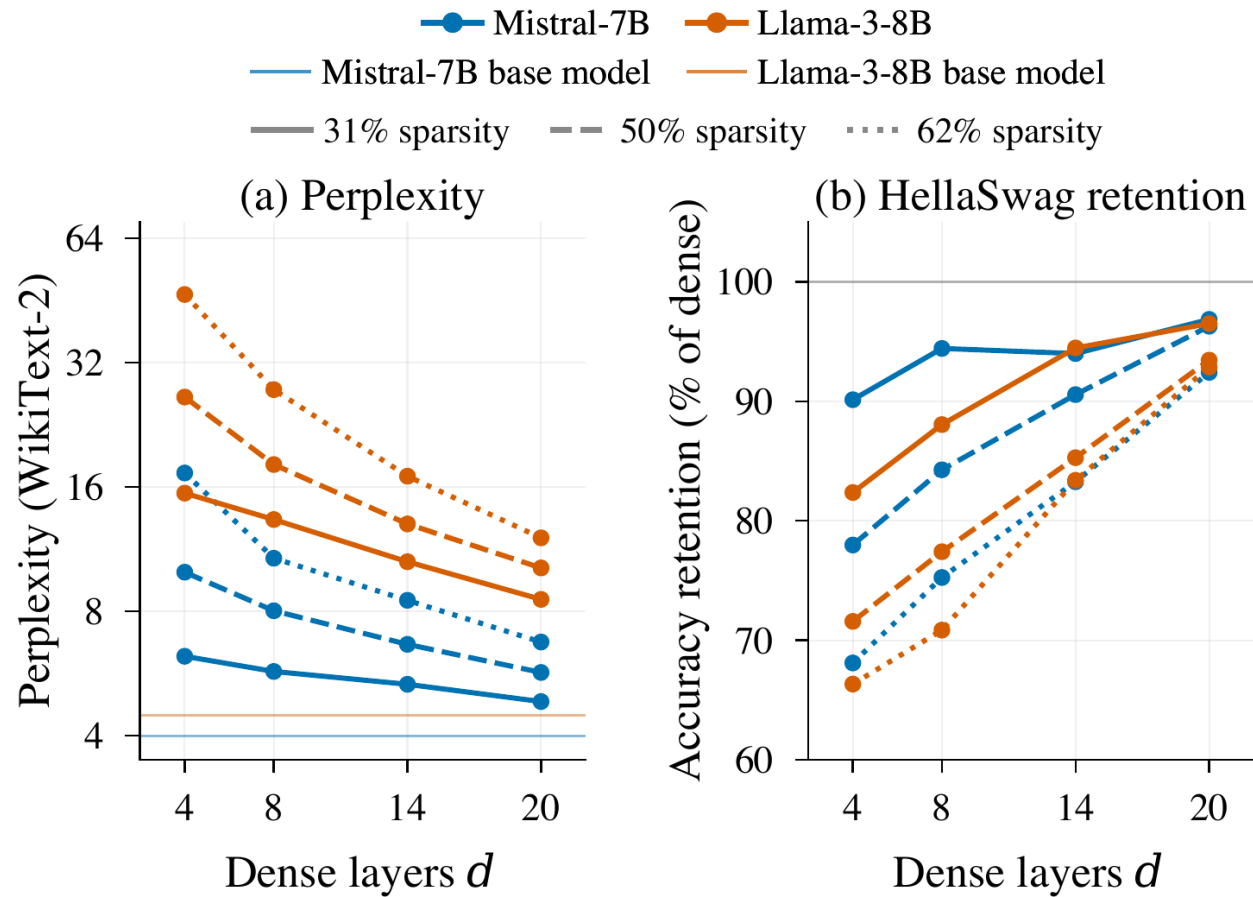
593 → 113 ms

per token: 9.0 → 14.8 GiB (d = 2 → 28)

I/O-bound

compute and predictor are small;
next gains are in I/O

Quality: the dense prefix is the knob



~99%

correct centroids: loss is almost all from clustering

92–96%

of dense HellaSwag accuracy at $d = 20$

No retraining

WikiText-2 centroids transfer to HellaSwag



Takeaways

NeuroPrefetcher · ICPP '26

1

Storage is a weight tier

When the model exceeds memory, manage storage traffic at the granularity of the rows each token needs

2

Predict once, fetch the delta

One-shot prediction after layer 0 plus 82–85% reuse across tokens → 7.9–12.0× over llama.cpp on Jetson AGX Orin

3

I/O is the next frontier

Deeper prefetch pipelines, stronger coalescing, quantized sparse-row formats, better overlap of I/O and compute

Thank you!

Questions?

Nobel Dhar • ndhar@students.kennesaw.edu

Supported in part by US NSF grants SHF-2210744, OAC-2602212 and ERI-2501978



Backup

Details for Q&A



Backup: memory levels

Kernel mem=	CUDA total (GiB)	CUDA free (GiB)
11G	10.69	9.02
12G	11.67	9.99
13G	12.65	11.02
14G	13.64	11.94
15G	14.62	13.19
16G	15.60	13.88
17G	16.56	14.81

- The Linux mem= boot parameter limits total memory
- On unified memory, “free” covers both CPU and GPU allocations of our process
- Mistral-7B FP16 (13.49 GiB) exceeds memory up to 13.2 GiB